LECTURER: MAJA POPOVIĆ

REINFORCEMENT LEARNING

REINFORCEMENT LEARNING
TOPIC OUTLINE

Introduction to Reinforcement Learning	1
Dynamic Programming and Markov Decision Processes	2
Bandit Problems	3
Q-Learning	4
Reinforcement Learning Approaches	5
Summary: Reinforcement Learning	6

UNIT 4

Q - LEARNING

Recommended sources:

- https://www.davidsilver.uk/teaching/

  on-line lectures (videos + slides)

- https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf

  on-line book

Q-learning

- control (policy improvement)
- model free approach
  -- time difference approach
- off-policy approach

Now:

- Revisions (value functions, prediction/control, on-/off-policy, model)
- Model-free prediction

Next time:

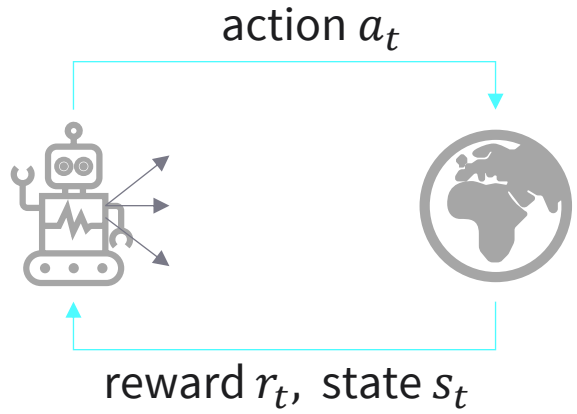
- Model-free control
  - Q-learning
- Model-based learning
- Function approximation

Revisions

(value functions, prediction/control, on/off policy, model)

RL: optimising computer decisions through trial-and-error learning

- **State:** current condition
- **Action:** move in each state
- **Reward:** feedback and incentive for actions
- **Policy:** decision-making strategy



- **Agent**: the learning entity
- **Environment**: external context

Source of the text: Sutton & Barto, 2018.
Source of the image: Own representation.

Model

the environment's behaviour

- next state:

$$P_{ss'}^{a} = P(S_{t+1} = s' \mid S_t = s, A_t = a)$$

- next reward:

$$R_s^a = E[\, R_{t+1} \mid S_t = s, A_t = a]$$

Prediction

- **evaluate** the future reward for **a given policy**

  policy evaluation

Control

- **optimise** the future reward **by finding the best policy**

  policy improvement (policy evaluation + policy update)

Control (policy improvement)

- **on-policy**

  evaluate the same policy which is used to generate actions

- **off-policy**

  two separate policy functions

  * behaviour policy (policy used to generate actions)
  * target policy (the one which is evaluated and improved)

  tries to evaluate greedily while following a more exploratory strategy

Value functions

state value function:

$v_\pi(s) = E_\pi[G_t \mid S_t = s]$

expected return starting from the state s and then following the policy π

action value function:

$q_\pi(s, a) = E_\pi[G_t \mid S_t = s, A_t = a]$

expected return starting from the state s, taking the action a, and then following the policy π

Value function: cumulative reward

cumulative reward, return

$G_t = R_{t+1} + \beta R_{t+2} + \beta^2 R_{t+3} + \cdots$

$\beta \in [0,1]$ -- discount factor

Bellman equation

decomposition into immediate reward and value function of the successor state

state value function:

$$v_\pi(s) = E_\pi[R_{t+1} + \beta v_\pi(S_{t+1}) \mid S_t = s]$$

action value function:

$$q_\pi(s, a) = E_\pi[R_{t+1} + \beta q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

Bellman expectation equation for $v_\pi(s)$

prediction

$$v_\pi(s) = \sum_{a \in A} \pi(a|s) \cdot q_\pi(s,a)$$

$$v_\pi(s) = \sum_{a \in A} \pi(a|s) \cdot \left(R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot v_\pi(s') \right)$$

Bellman expectation equation for $q_\pi(s)$

prediction

$$q_\pi(s,a) = R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot v_\pi(s')$$

$$q_\pi(s,a) = R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \sum_{a \in A} \pi(a'|s') \cdot q_\pi(s',a')$$

Optimal value function

control

$$v_*(s) = \max_\pi v_\pi(s)$$

$$q_*(s,a) = \max_\pi q_\pi(s,a)$$

the maximal value function over all policies π

a MDP is “solved” if the optimal value function is known

Bellman optimality equation

control

$$v_*(s) = \max_a \left(R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot v_*(s') \right)$$

$$q_*(s,a) = R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot \max_{a'} q_*(s',a')$$

$$\sum_{a \in A} \pi(a|s) \Rightarrow \max_a$$

control (max) and prediction (Σ)

$$v_*(s) = \max_a \left(R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot v_*(s') \right)$$

$$v_\pi(s) = \sum_{a \in A} \pi(a|s) \cdot \left(R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot v_\pi(s') \right)$$

$$q_\pi(s,a) = R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \sum_{a \in A} \pi(a'|s') \cdot q_\pi(s',a')$$

$$q_*(s,a) = R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot \max_{a'} q_*(s',a')$$

Bellman optimality equation

control

- non-linear
- no closed-form solution

- many iterative solution methods:
  - policy iteration
  - value iteration
  - Q-learning
  - SARSA

Dynamic programming: overview

problem	Bellman equation	Algorithm
prediction	Bellman expectation equation	policy evaluation
control	Bellman expectation equation + greedy policy improvement	policy iteration
control	Bellman optimality equation	value iteration

ALGORITHMS FOR THE OPTIMAL CONTROL OF THE MDP

Policy Iteration
The Actor – Critic Method

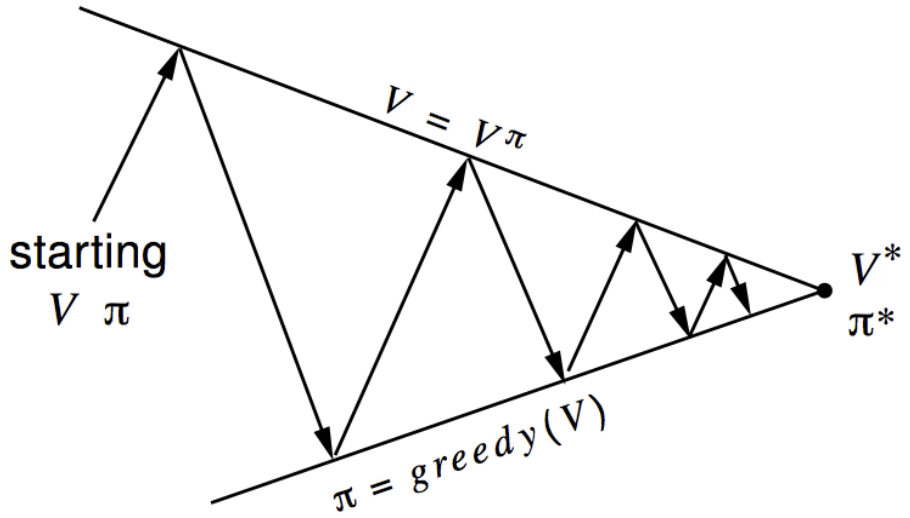
- Policy evaluation **(Critic)**
- Policy improvement **(Actor)**
- Terminate when optimal policy is found

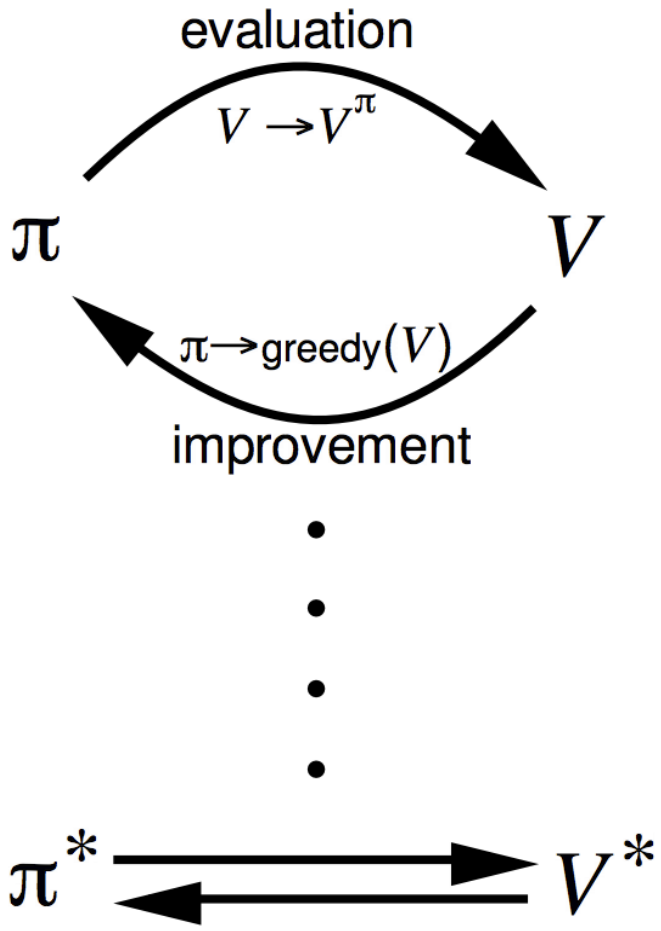
Value Iteration
Iterative Optimisation

- Initialise values for each state
- Update values using Bellman's equation
- Calculate new values based on maximal reward
- Converge to optimal value function

Source of the text: Sutton and Barto, 2018.

Generalised policy iteration





- iterative policy evaluation
  (estimate v_π)
- greedy policy improvement
  (find $\pi' > \pi$)

Types of RL agents

Value-based	Policy-based	Actor critic
no policy	policy	policy
value function	*no value function*	value function

Model-based	Model-free
policy and/or value function	policy and/or value function
model	*no model*

Prediction and control methods

- Dynamic programming

  estimate/optimise the value function of a **known** MDP

- Model-free methods

  estimate/optimise the value function of an **unknown** MDP

- Model-based methods

  estimate the model of an unknown MDP

  then use the estimated model to estimate/optimise the value function

Model-free and model-based RL

- Model-free RL (prediction or control)
  - learning value function and/or policy from experience
  - no model

- Model-based RL (prediction or control)
  - learning model from experience
  - planning value function and/or policy from model

Model

the environment's behaviour

- next state:

$$P_{ss'}^{a} = P(S_{t+1} = s' \mid S_t = s, A_t = a)$$

- next (immediate) reward:

$$R_s^a = E[\, R_{t+1} \mid S_t = s, A_t = a]$$

Model-free RL

Model-free RL

- Monte Carlo learning
- Temporal difference (TD) learning
    - prediction
    - control
        - on-policy
        - off-policy

Monte Carlo RL

- no model (transitions and rewards)
- learns from episodes of experience
- learns from **complete** episodes

  no dependance on previously updated values (“**bootstrapping**”)
- **only works** for **episodic** MDPs (all episodes must terminate)

- uses the simplest possible idea: **value** = **average return**

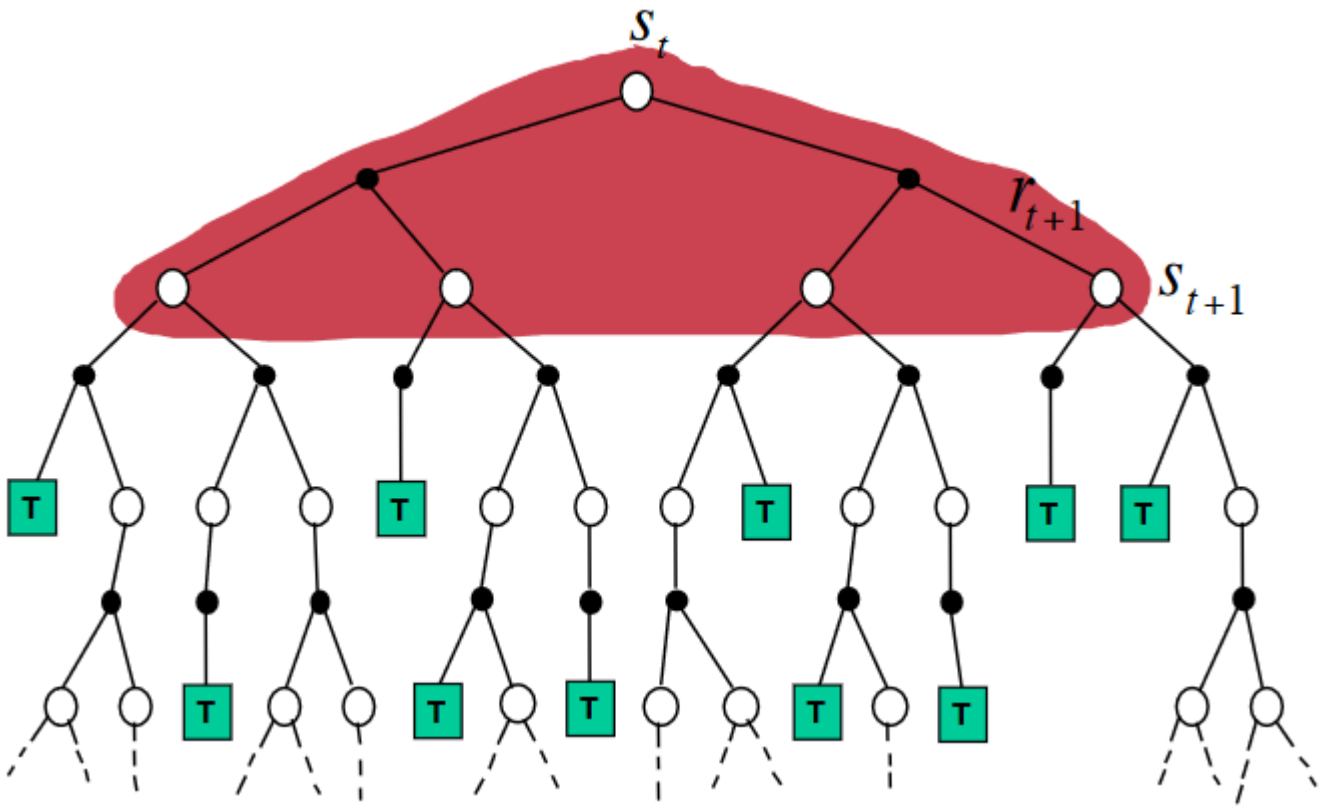
Temporal difference RL

- no model (transitions and rewards)
- Learns from episodes of experience
- Learns from **incomplete** episodes

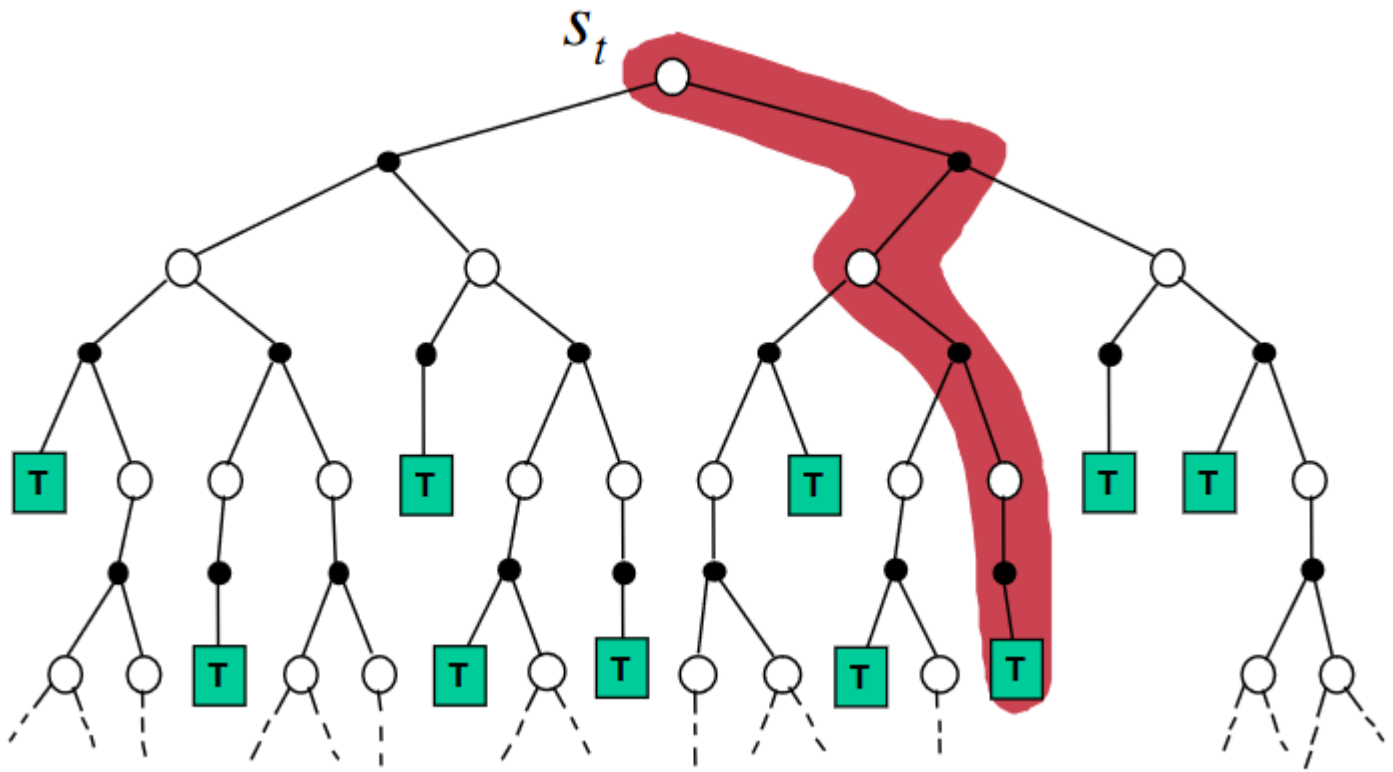
  by "bootstrapping" (using previous updates)
- Works for **continuing** MDPs (no need for episodes to terminate)

- Updates an estimate towards an estimate (return not known before the final state)

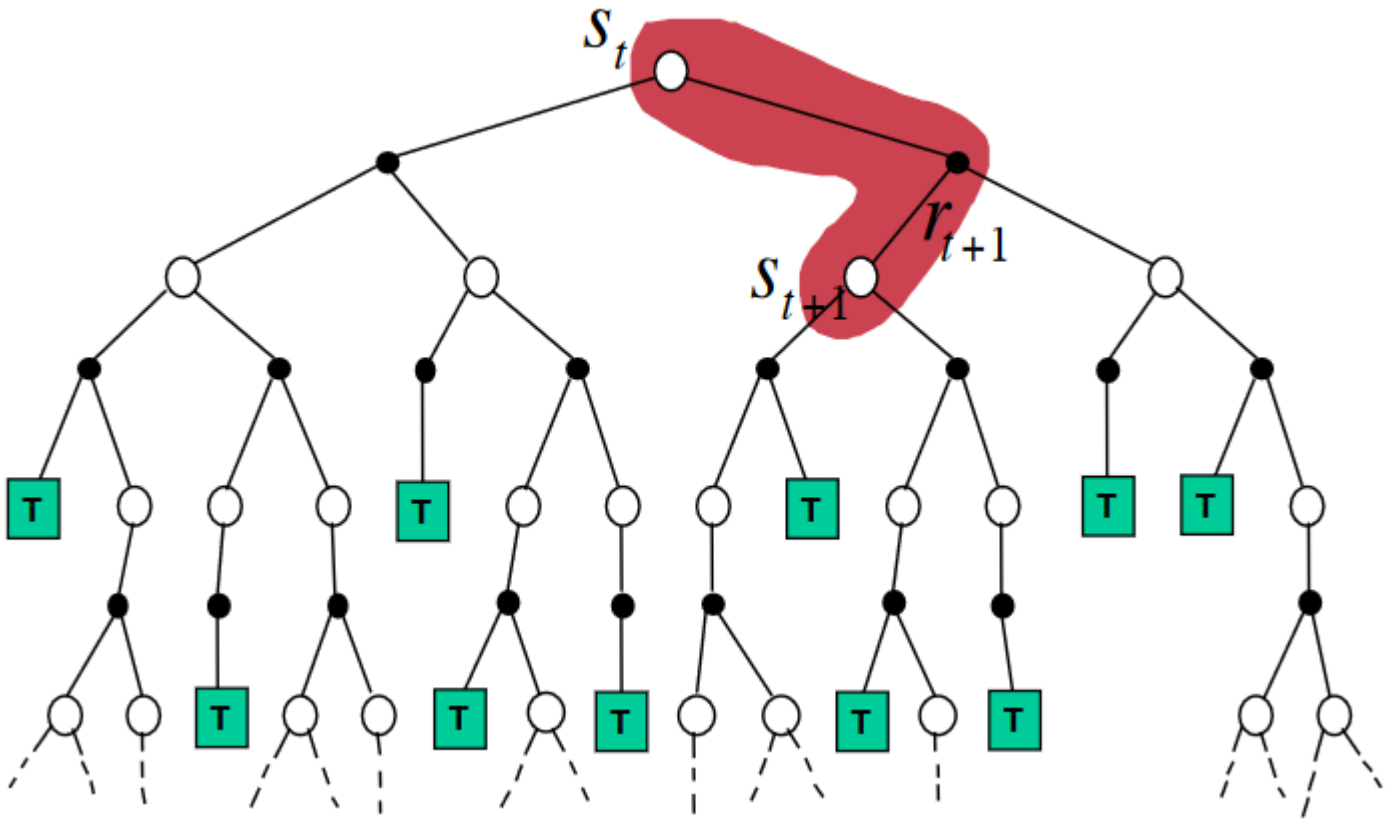
Dynamic Programming



Monte Carlo



Temporal difference



Model-free prediction

Monte Carlo prediction (MC)

Monte Carlo prediction: state value function

estimating $v_\pi(s)$:

$$v_\pi(s) = E_\pi[G_t \mid S_t = s]$$

from episodes of experience (until a time step T) under policy π:

$$S_1 A_1 R_2 S_2 A_2 \ldots R_T S_T$$

- return: $G_t = R_{t+1} + \beta R_{t+2} + \beta^2 R_{t+3} + \beta^3 R_{t+4} + \cdots + \beta^{T-1} R_T$
- estimate: $V(s)$ – *empirical mean* return instead of *expected* return

Monte Carlo prediction: state value function

$$V(s) = \frac{1}{N(s)} \sum_{t=1}^{T} G_t \cdot \delta(S_t, s)$$

- $N(s) =$ number of time steps where the state s is visited
  (number of occurrences of s in the episode)
- $G_t =$ return at each time step
- $\delta(S_t, s) = 1$ if $S_t = s$ (0 otherwise)

$$\lim_{N(s)\to\infty} V(s) = v_\pi(s)$$

Monte Carlo prediction: action value function

$$Q(s,a) = \frac{1}{N(s,a)} \sum_{t=1}^{T} G_t \cdot \delta(S_t, s) \cdot \delta(A_t, a)$$

- $N(s,a) =$ number of time steps where the state s is visited and action a is taken (number of occurrences of (s,a) in the episode)
- $G_t =$ return at each time step
- $\delta(S_t, s) = 1$ if $S_t = s$ (0 otherwise)
- $\delta(A_t, a) = 1$ if $A_t = a$ (0 otherwise

$$\lim_{N(s)\to\infty} Q(s,a) = q_\pi(s,a)$$

Monte Carlo prediction: action value function

$$Q(s,a) = \frac{1}{N(s,a)} \sum_{t=1}^{T} G_t \cdot \delta(S_t, s) \cdot \delta(A_t, a)$$

possible problem: some state-action pairs might never be visited

but all possible actions in a state should be evaluated

- "exploration starts"

  every (q, a) pair should have a non-zero probability to be selected as a start
- or consider only stochastic policies with non-zero probabilities for all actions

Monte Carlo prediction

- “first visit”
  - follows the steps only after the first visit (occurrence) of state s in an episode
  - return samples are independent (obtained from different episodes)

- “every visit”
  - follows the steps after each visit (occurrence) of state s in an episode
  - return samples are non-independent (might be obtained from same episodes)

Monte Carlo incremental prediction

Incremental mean μ_k of a sequence $x_1, x_2, \ldots$:

$$\mu_k = \frac{1}{k}\sum_{n=1}^{k} x_n = \frac{1}{k}\left(x_k + \sum_{n=1}^{k-1} x_n\right)$$

$$= \frac{1}{k}(x_k + (k-1)\mu_{k-1})$$

$$= \mu_{k-1} + \frac{1}{k}(x_k - \mu_{k-1})$$

Monte Carlo incremental prediction

Incremental Monte Carlo update:

$$V(S_t) = V(S_t) + \frac{1}{N(S_t)}(G_t - V(S_t))$$

or

$$V(S_t) = V(S_t) + \alpha(G_t - V(S_t))$$

α = step size

Temporal Difference prediction (TD)

Temporal difference prediction

incremental Monte Carlo update:

$$V(S_t) = V(S_t) + \alpha(\boldsymbol{G_t} - V(S_t))$$

simplest temporal difference update:

$$V(S_t) = V(S_t) + \alpha(\boldsymbol{R_{t+1} + \beta V(S_{t+1})} - V(S_t))$$

G_t = actual return

$R_{t+1} + \beta V(S_{t+1})$ = estimated return

Temporal difference prediction

$$V(S_t) = V(S_t) + \alpha(R_{t+1} + \beta V(S_{t+1}) - V(S_t))$$

$R_{t+1} + \beta V(S_{t+1})$ = TD target

$R_{t+1} + \beta V(S_{t+1}) - V(S_t) = \delta_t$ = TD error

TD vs MC (1)

TD can learn from incomplete sequences, without the final outcome

- can learn on-line after every step
- works in continuing (non-terminating) environments

MC requires complete sequences and the final outcome

- must wait until the end of the episode
- works only in episodic (terminating) environments

TD vs MC (2)

TD has some bias, low variance

- true TD target $R_{t+1} + \beta v_\pi(S_{t+1})$ is *unbiased* estimate of $v_\pi(S_t)$

  ! TD target $R_{t+1} + \beta V(S_{t+1})$ is *biased* estimate of $v_\pi(S_t)$
- depends on *one* action, transition, reward ⇒ *low variance*

MC has no bias, high variance

- return $G_t = R_{t+1} + \beta R_{t+2} + \cdots + \beta^{T-1} R_T$ is *unbiased* estimate of $v_\pi(S_t)$
- depends on *many* actions, transitions, rewards ⇒ *high variance*

TD vs MC (3)

TD exploits the Markov property

- future states depend on the current state
- more effective in Markov environments

MC does not exploit the Markov property

- based on the entire process
- more effective in non-Markov environments

Bootstrapping and sampling

Bootstrapping -- update based on another updates

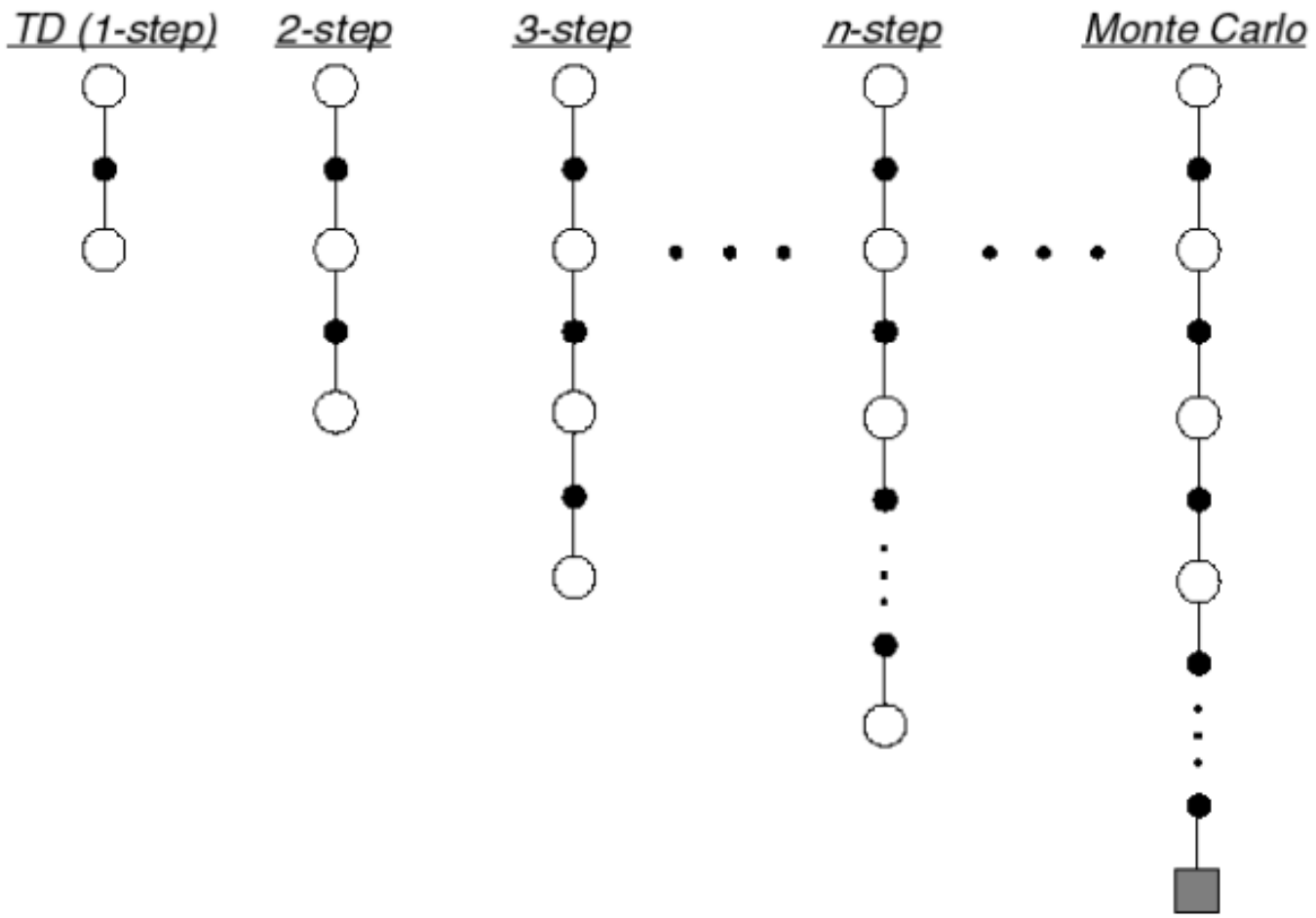
- DP: yes
- MC: no
- TD: yes

Sampling -- update based on samples

- DP: no
- MC: yes
- TD: yes

between MC and TD:
n-step prediction and TD(λ)

n-step TD



until now:

$n = 1$

n-step return

$$n = 1 \quad G_t^{(1)} = R_{t+1} + \beta V(S_{t+1}) \qquad \text{(TD)}$$

$$n = 2 \quad G_t^{(2)} = R_{t+1} + \beta R_{t+2} + \beta^2 V(S_{t+2})$$

$$n = 3 \quad G_t^{(3)} = R_{t+1} + \beta R_{t+2} + \beta^2 R_{t+3} + \beta^3 V(S_{t+3})$$

$$\cdots$$

$$n = \infty \quad G_t^{(\infty)} = R_{t+1} + \beta R_{t+2} + \cdots + \beta^{T-1} V(S_T) \qquad \text{(MC)}$$

n-step TD

- n-step return:

$$G_t^{(n)} = R_{t+1} + \beta R_{t+2} + \beta^2 R_{t+3} + \cdots + \beta^{n-1} R_{t+n} + \beta^n V(S_{t+n})$$

$t + n =$ horizon of the n-step return

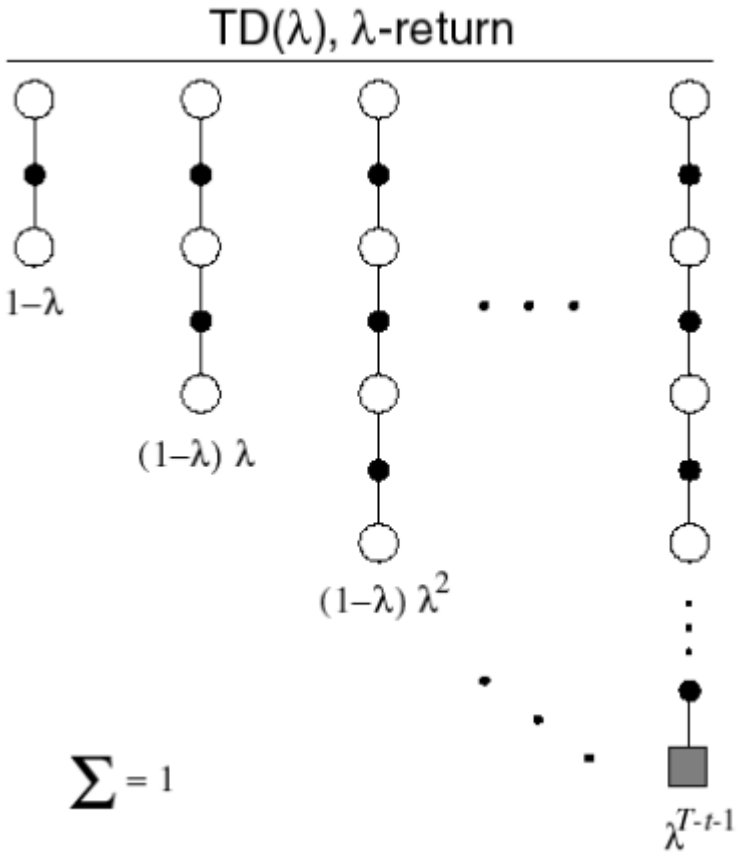
➢ n-step temporal difference learning:

$$V(S_t) = V(S_t) + \alpha \left(G_t^{(n)} - V(S_t) \right)$$

λ return

combines all n-step returns $G_t^{(n)}$
with weight $(1-\lambda)\lambda^{n-1}$

$$G_t^{\lambda} = (1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}G_t^{(n)}$$



TD(λ)

- forward view

$$V(S_t) = V(S_t) + \alpha\left(G_t^{\lambda} - V(S_t)\right)$$

- backward view

$$V(s) = V(s) + \alpha\delta_t E_t(s)$$

δ_t = TD error

$E_t(s)$ = eligibility trace

TD(λ)

- forward view

$$V(S_t) = V(S_t) + \alpha\left(G_t^{\lambda} - V(S_t)\right)$$

-- looks into the future

-- only works for complete episodes

TD(λ)

- backward view

$$V(S_t) = V(S_t) + \alpha\delta_t E_t(S_t)$$

-- looks into the past

-- updates on-line (every step) from incomplete sequences

- $\delta_t = R_{t+1} + \beta V(S_{t+1}) - V(S_t)$ = TD error
- $E_t(s)$ = eligibility trace

Eligibility traces

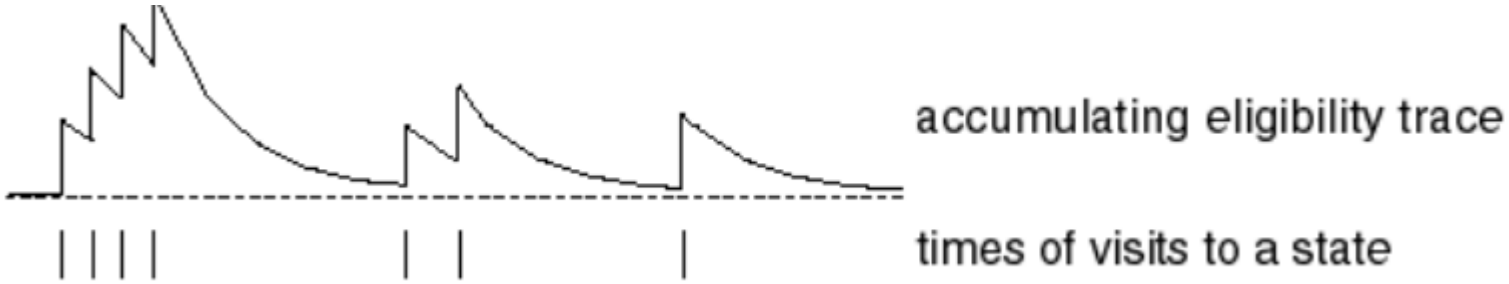


combined influence of

- most frequent states
- most recent states

(takes both bells and light into account)

$$E_0(S_t) = 0$$
$$E_t(S_t) = \beta\lambda E_{t-1}(S_t) + 1$$



for a particular state s: $E_t(s) = \beta\lambda E_{t-1}(s) + \delta(S_t, s)$

Model-free control

Model-free RL

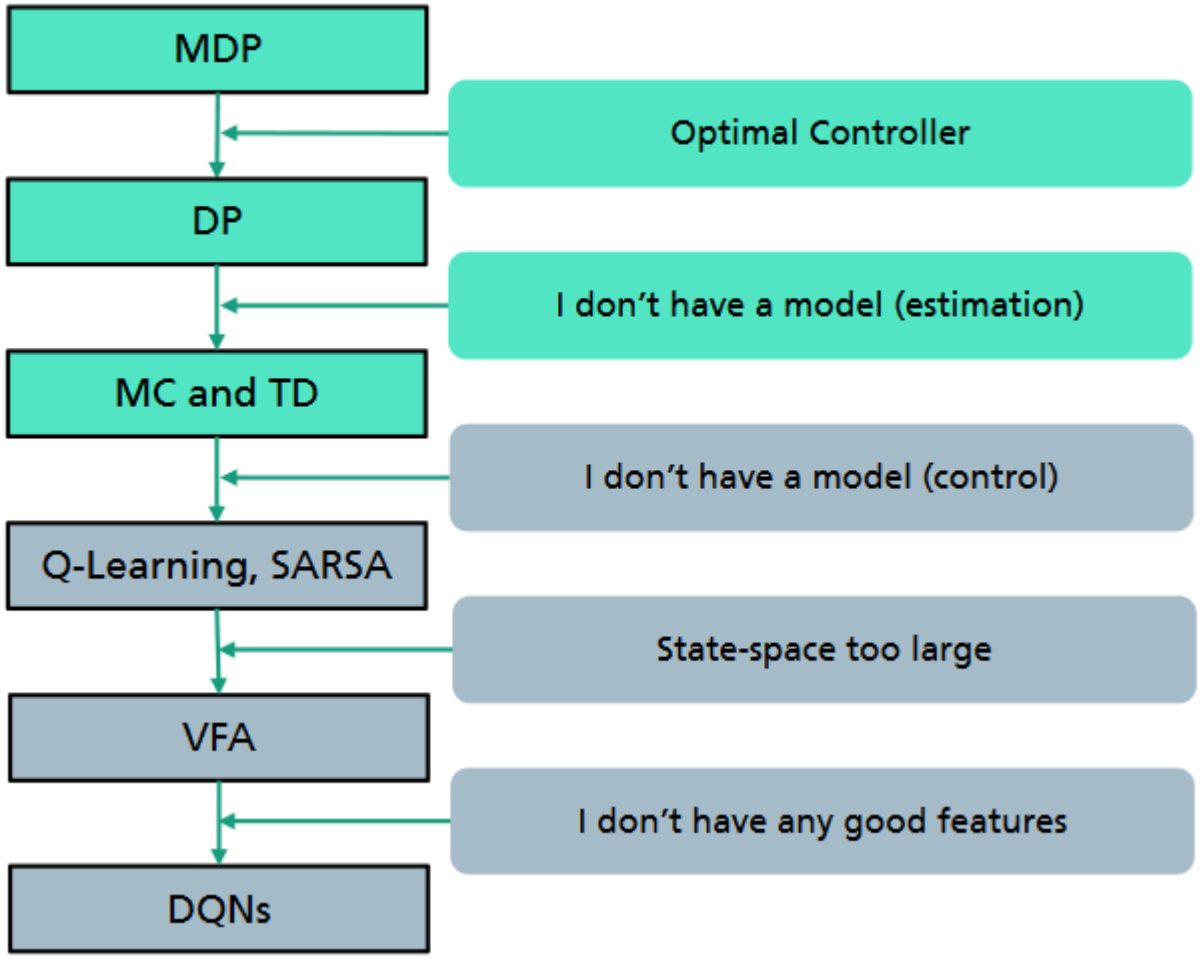
- Monte Carlo learning
- Temporal difference (TD) learning
    - prediction
    - control
        - on-policy
        - off-policy

Model-free control

- on-policy
  - Monte Carlo
  - temporal difference (SARSA)
- off-policy
  - Monte Carlo
  - temporal difference (Q-learning)

Questions?

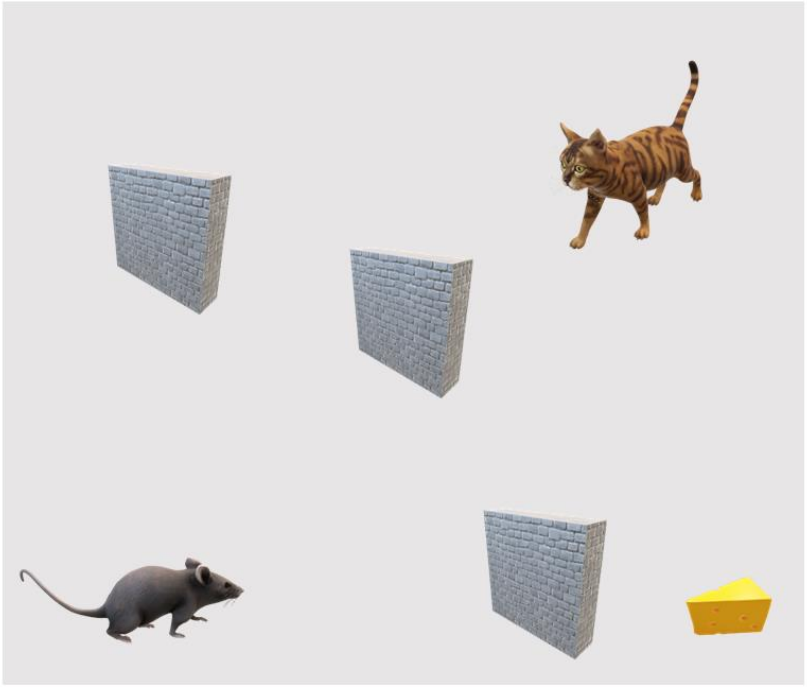
REINFORCEMENT LEARNING



Game context and rules

- Cat and mouse move with 8 degrees of freedom
- Mouse avoids cat and reaches cheese
- Cat catches mouse, score changes, new round starts

Cat and Mouse Game





Considering uncertain cat movements, how does mouse reach the cheese?

Source of the image: Own representation.

- **Q-factor:** reward when taking optimal action following optimal policy

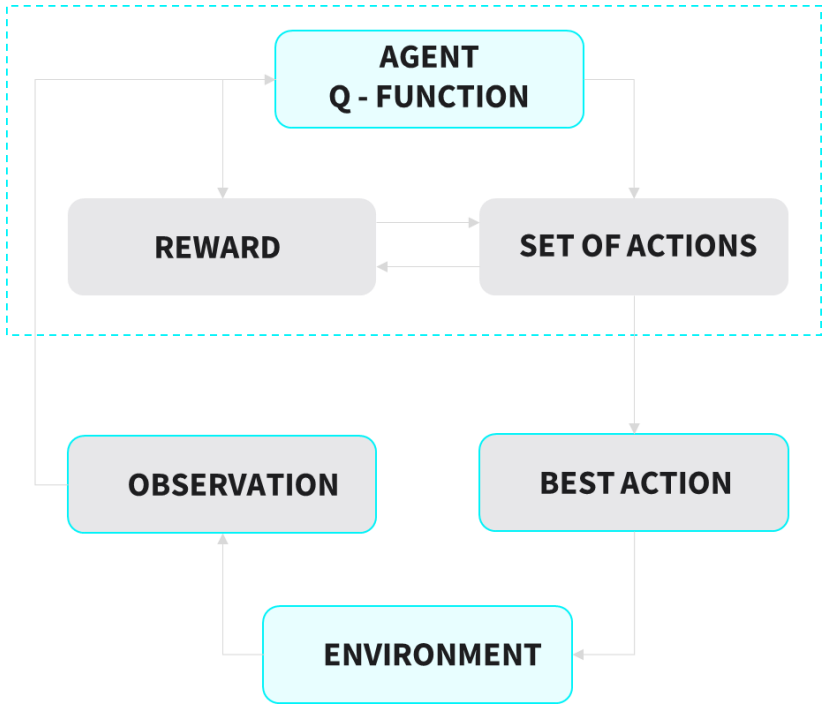
$$Q_1(s,a) \equiv r\,(s,a)$$

$$\pi_1(s) = \underset{\alpha}{\operatorname{argmax}}\, Q_1(s,a)$$

- Convergence assures optimal value function (Bellman equation)

$$V^*(s) = \max_{\alpha}\left[r(s,a) + \gamma \sum_{s'} P(s'\,|s,a)\, V^*(s')\right]$$

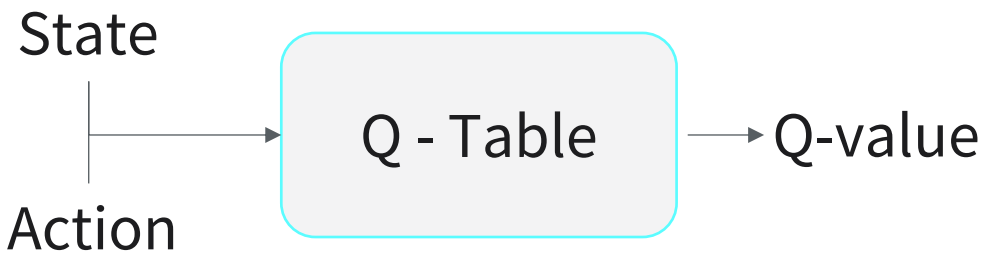
Goal: discover optimal action strategy through iterative value updates



Source of the text: Sutton & Barto, 2018.
Source of the image: Own representation.

Q-LEARNING ALGORITHM

Learn optimal actions through iterative Q-value updates and exploration



1. Initialize $Q(s,\ a)$ arbitrarily
2. Start at initial state s
3. Repeat until terminal state reached
   - Choose action α for state s
   - Take action α, observe reward r, and move to new state s'
   - Update Q-values

$$Q(s,a) = Q(s,a) +$$

$$\alpha\left[r + \gamma\ \max_{\alpha} Q(s',a) - Q(s,a)\right]$$

4. Set the current state to new state $s \mapsto s'$

Source of the text: Sutton & Barto, 2018

SARSA ALGORITHM

- **Q-learning (off policy)**
  values updated independently of current policy

- **SARSA (on policy)**
  next action follows same policy

1. Initialize $Q(s,\ a)$ arbitrarily
2. Start at initial state s
3. Repeat until terminal state reached
   - Choose action α for state s
   - Take action α, observe reward r, and move to new state s'
   - Choose action a' for new state s'
   - Update Q-values

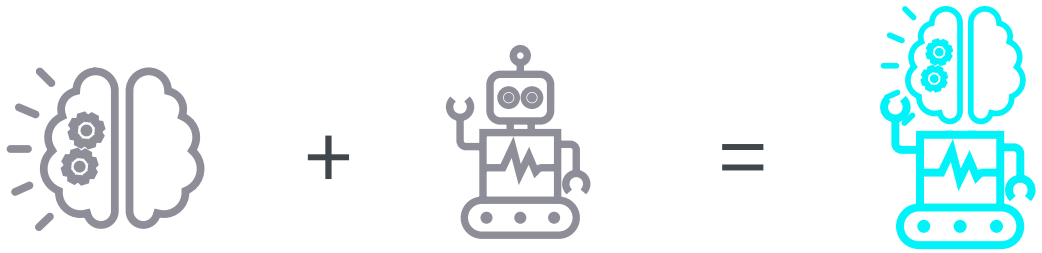
$$Q(s,a) = Q(s,a) +$$

$$\alpha\left[r + \gamma\ \max_{\alpha} Q(s',a') - Q(s,a)\right]$$

4. Set current state to new state $s \mapsto s'$ and current action to new action $\alpha \mapsto \alpha'$

Source of the text: Sutton & Barto, 2018

Synergy of neural networks and reinforcement learning



Neural Network

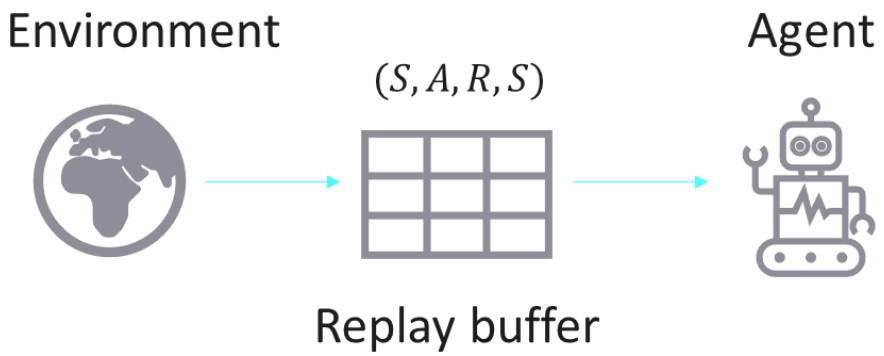
Reinforcement Learning

Deep Reinforcement Learning

- Impractical to build Q-table explicitly
- State-action space grows too big
- Approximate Q-function using neural networks
- Learn Q-values from state – action pairs

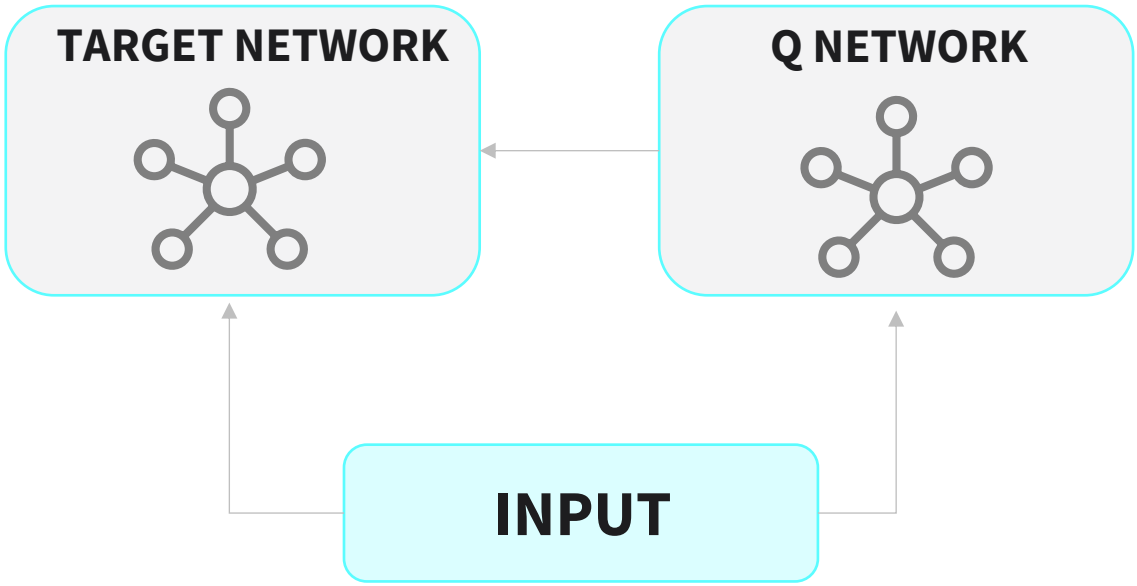
Experience Replay

- Save transitions $(S_t, A_t, R_{t+1}, S_{t+1})$ into buffer and sample batch
- Use batch to train agent



Double Q-Learning

- Utilize separate target and value networks

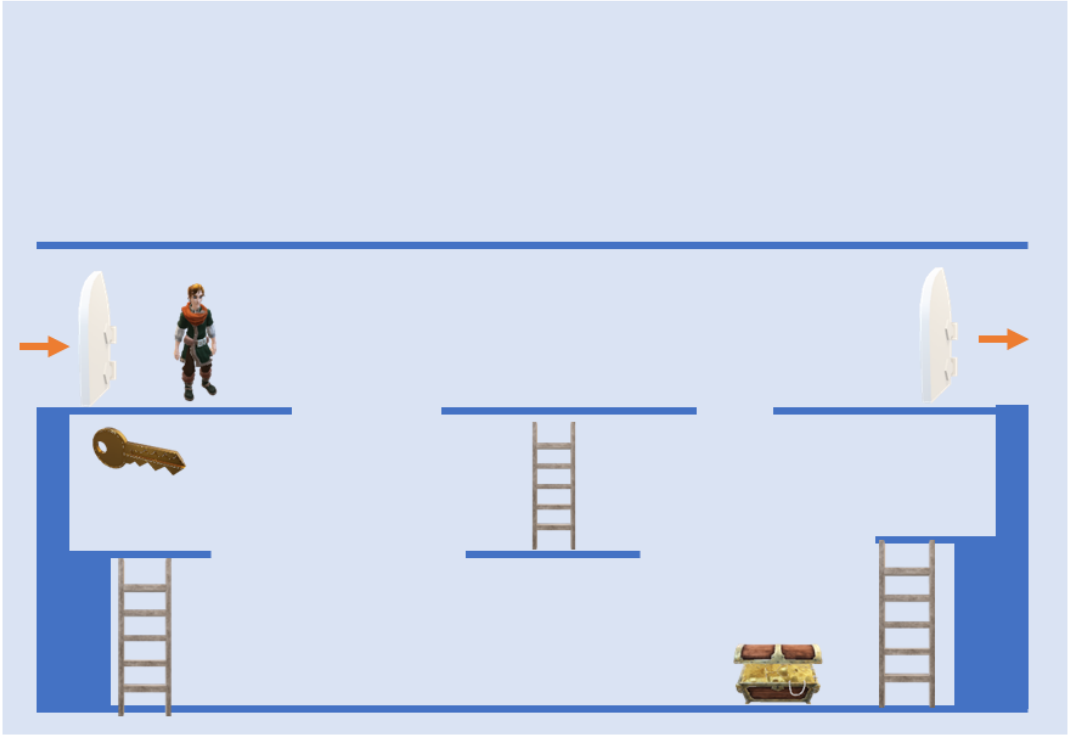


Delayed sparse rewards

- Hard learning when rewards delayed or sparse
- Exploration challenging, requiring new strategies

Hierarchical learning

- Complex environments: multiple objectives, interdependencies
- Hierarchy: meta-controller sets goals, sub-controller achieves them

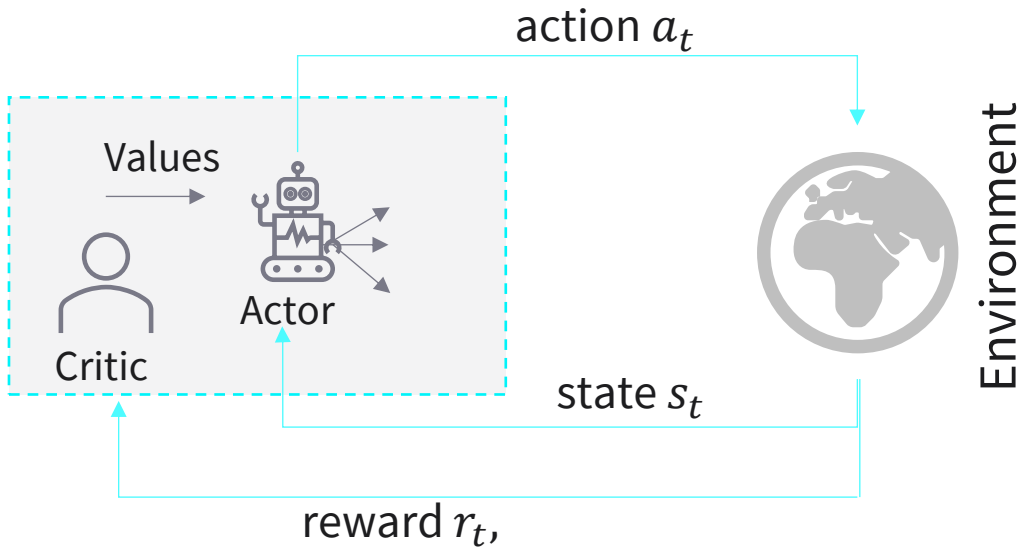


Layout of a complex games with multiple sub-goals

Source of the image: Own representation.

Simultaneous policy evaluation and improvement
Combine policy-based and value-based learning

- **Actor** learn policy (**action selection**)
- **Critic**: estimate value function (**policy evaluation**)
- Enhances stability, convergence, and exploration



Source of the image: Own representation.

SESSION 4

TRANSFER TASK



Case study

Your task is to develop an AI-agent that can play the Atari game Pong using deep Q-learning. Your goal is to develop an optimal policy that maximizes the game score



Task

What key steps will you consider when implementing Q-learning? Which neural network architecture would be most appropriate? How to optimize Q-learning? Consider factors such as experience replay and target network updates, among others

TRANSFER TASK
PRESENTATION OF THE RESULTS

Please present your results.

The results will be discussed in plenary.





1. Suppose we are using policy iteration to find the optimal policy of MDP. This results in a sequence of policy functions $\varphi[1], \varphi[2], \ldots, \varphi[n]$. After the nth iteration we notice that $\varphi[n-1] \neq \varphi[n]$, but that the policy value functions of these two policies are equal. What should we conclude?
   a) An optimal policy doesn‘t exist
   b) We have not converged to an optimal policy
   c) $\varphi[n]$ is an optimal policy but $\varphi[n-1]$ is not
   d) Both $\varphi[n-1]$ and $\varphi[n]$ are optimal policies



2. A minimum discounted cost MDP has a state space $\chi = \{1,2,3,\ldots,100\}$. From state $x = 2$ two actions are available. If action $a = 1$ is chosen, the process transitions to states $x = 1$ or $x = 3$ with probability 1/2 each, and a stage cost $R(2,1) = 5$ is accrued. If action $a = 2$ is chosen, the process transitions to state $x = 1$ with probability once, and a stage cost of $R(2,2) = 10$ is accrued. The discount factor is $\beta = 0.9$. Then suppose policy iteration is used to evaluate the optimal policy. The policy value function for the nth policy iterate $\varphi[n]$ has a policy value function with values $V'(1) = 20.5$, $V'(3) = 36.4$. What action will the next iterate $\varphi[n+1]$ choose from state $x = 2$? Justify your answer

   a) Action a = 1 only
   b) Action a = 2 only
   c) Both actions are equally optimal
   d) There is not enough information to determine the optimal action



3. A multilayer perceptron has an input layer with 5 nodes, a hidden layer with 16 nodes, and an output layer with 2 nodes. What is the total number of weight and bias parameters
   a) 23
   b) 41
   c) 130
   d) 178

LIST OF SOURCES

Sutton, R. S., & Barto, A. G. (2018a). Reinforcement learning: An introduction (2nd ed.). MIT press.

© 2022 IU Internationale Hochschule GmbH
This content is protected by copyright. All rights reserved.
This content may not be reproduced and/or electronically edited, duplicated, or distributed in any kind of form without written permission by the IU Internationale Hochschule GmbH.